K. L. E. SOCIETY'S

K. L. E. INSTITUTE OF TECHNOLOGY,

Opp. Airport, Gokul, Hubballi-580 027

Phone: 0836-2232681 Website: www.kleit.ac.in

A Mini Project Report on

"Sorting Algorithm Visualizer: Quick Sort & Merge Sort"

Computer Science and Engineering Semester IV

Batch 16

Analysis and Design of Algorithm (BCS401)

Academic Year 2025-26

Submitted By

Ms. Annapoorna. Hasabi (2KE24CS017)

Mr. Jadhav. Yuvraj. Laxman (2KE24CS047)

Ms. Jagruti. Narvekar (2KE24CS048)

Ms. Kavana. Araladinni (2KE24CS055)

Under the Guidance of

Assistant Prof. Shanta. Kallur

Department of Computer Science & Engineering

Department of Computer Science
K.L.E. Institute of Technology, Hubballi-27
2025-2026

CERTIFICATE

This is to certify that the Mini Project work titled **"Sorting Algorithm Visualizer: Quick Sort & Merge Sort"** is a bonafide work carried out by **Ms. Annapoorna. Hasabi (2KE24CS017), Mr. Jadhav. Yuvraj. Laxman (2KE24CS047), Ms. Jagruti. Narvekar (2KE24CS048), Ms. Kavana. Araladinni (2KE24CS055)** in partial fulfilment for the award of degree of **Bachelor of Engineering** in **IV Semester, Computer Science Engineering, K.L.E. Institute of Technology** under the **Visvesvaraya Technological University, Belagavi** during the year **2025- 2026**. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The seminar report has been approved as it satisfies the academic requirements in respect of seminar work prescribed for the said degree.

Signature of the Guide
(Mrs. Shanta. Kallur)

ACKNOWLEDGEMENT

The mini project report on **"Sorting Algorithm Visualizer: Quick Sort & Merge Sort"** is the outcome of guidance, moral support and devotion bestowed on me throughout my work. For this I acknowledge and express my profound sense of gratitude and thanks to everybody who have been a source of inspiration during the project work.

First and foremost I offer my sincere phrases of thanks with innate humility to our Principal **Dr. Manu T.M** who has been a constant source of support and encouragement. I feel deeply indebted to our H.O.D. **Dr. Rajesh Yakkundimath** for the right help provided from the time of inception till date. I would take this opportunity to acknowledge our Guide **Assistant Prof. Shanta. Kallur** who not only stood by us as a source of inspiration, but also dedicated his/her time for me to enable me to present the project on time.

Last but not least, I would like to thank my parents, friends & well wishers who have helped me in this work.

Name of the students

1. Annapoorna. Hasabi
2. Jadhav. Yuvraj. Laxman
3. Jagruti. Narvekar
4. Kavana. Araladinni

PATIENT APPOINTMENT SCHEDULING SYSTEM

A DATABASE MANAGEMENT SYSTEM PROJECT REPORT

Submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY
IN COMPUTER SCIENCE & ENGINEERING

DEVELOPED BY:
[Student Name / Roll Number]

UNDER THE GUIDANCE OF:
[Instructor / Guide Name]

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
UNIVERSITY NAME / INSTITUTE DETAILS

Session: 2025 - 2026

TABLE OF CONTENTS

Chapter 1
INTRODUCTION

1.1 Introduction

Modern medical facilities and clinics face significant operational hurdles when managing patient appointments, consultation slots, and physician availability using manual, paper-based records. Manual processes frequently lead to scheduling overlapping appointments (double-bookings), patient queue blockages, high administrator stress, and structural record-keeping errors. This lack of coordination deteriorates the patient experience and reduces healthcare delivery efficiency.

To address these operational deficits, this Database Management System (DBMS) project introduces 'MediFlow', a dedicated Patient Appointment Scheduling System. MediFlow utilizes a three-tier software model that encapsulates clinic logical schemas directly inside a secure Relational Database Management System. By mapping real-world medical entities (patients, doctors, logs, and summaries) into a normalized MySQL schema, clinic admins can automate appointment queues, track historical records, and ensure absolute consistency.

1.2 Objective

The fundamental objective of this database application is to deliver a reliable, automated, and conflict-free tool for medical schedule management. Key database-specific design targets include:

* **Integrity and Non-Overlapping Scheduling:** Enforce relational foreign keys and transaction boundaries so that patients and doctors cannot double-book the same slot.
* **Encapsulation of Query Joins:** Create an SQL Virtual View (`upcoming_appointments_view`) to handle multi-table joins inside the database engine, returning a clean upcoming schedule to APIs.
* **Automated Security Audit Trails:** Use database triggers to intercept appointment changes (inserts and status updates) and write immutable log audits directly to an independent logs table.
* **Row-by-Row Cursor Processing:** Utilize a procedural SQL database cursor to loop through daily appointments on demand, compile a textual summary, and save it in a reporting structure.
* **Modern Architecture:** Build an easy-to-use modern web interface communicating with a fast asynchronous Node.js Express API server and a MySQL database backend.

1.3 Module Breakdown

The architecture of MediFlow is split into five distinct functional modules:

* **Patient Registration Module:** Enables the registration of new patients. Validates emails to prevent duplicates and writes patient profiles (name, DOB, gender, phone, email) to the database.

* **Doctor Specialty Lookup Module:** Maintains records of available specialist doctors. Allows frontend lookups to filter slots and query matching specialist consultants.

* **Appointment Booking Engine:** Manages scheduling slots. Feeds new appointment rows into MySQL while checking database constraint requirements.

* **Automated Audit Module:** Contains database-level triggers (`after_appointment_insert`, `after_appointment_update`) running completely independent of application code to log scheduling actions.

* **Daily Report Compilation Module:** Implements the stored procedure `generate_daily_report(date)`. Activates a cursor query that crawls specific dates, merges patient/doctor descriptions, and outputs administrative reports.

Chapter 2

SURVEY OF TECHNOLOGIES

2.1 Software Description

The software architecture follows a modular Three-Tier client-server pattern. The client layer runs directly inside any standard web browser, sending AJAX fetch commands. The middle tier runs a lightweight Node.js Express server to handle routing, input validation, and connection pool query dispatching. The storage tier uses MySQL to execute transactional data procedures. By cleanly decoupling client rendering, API controllers, and database storage, the application remains scalable and highly maintainable.

2.2 Languages & Architectures

2.2.1 HTML (HyperText Markup Language)

HTML5 is utilized to layout the application layout. The layout features a Sidebar Navigation panel, an active Status indicator checking database connectivity, a Patient Portal (registration modal, selector list, booking form), a Doctor Queue panel (for status management), and a Developer Lab interface exposing raw table counts, live database trigger log captures, and stored procedure execution outputs. All forms include HTML5 constraints (required fields, date range limits) to prevent malformed data payloads.

2.2.2 CSS (Cascading Style Sheets)

The visual aesthetics use premium CSS3 techniques, rendering a state-of-the-art administrative dashboard. The visual theme uses a sleek Dark Mode scheme with deep purple, indigo, and teal neon gradients. Glassmorphism is implemented via CSS backdrop-filters, rendering translucent cards. Interaction feedback is provided through CSS micro-animations on hover states, pulsing indicators, and sliding tabs. Layouts are completely responsive, adjusting cleanly using CSS Flexbox and CSS Grid frameworks.

2.2.3 PHP (Traditional Stack Survey)

Traditionally, academic DBMS projects are structured around PHP, a server-side scripting language running on Apache in XAMPP. In a standard PHP model, the server processes SQL queries sequentially via standard drivers (`PDO` or `mysqli`) and injects variables directly into HTML templates before responding. While PHP remains a robust and reliable stack, its synchronous model blocks threads for long-running database requests. In our modern implementation, Node.js replaces PHP as the application layer, running an asynchronous, event-driven event loop that uses non-blocking database pools to handle thousands of concurrent queries without blocking.

2.2.4 MySQL Database

MySQL is an open-source Relational Database Management System (RDBMS) configured using the transactional InnoDB storage engine. MySQL executes physical database operations. It maintains strict

constraints: it prevents orphan records using foreign keys with cascading deletions, exposes a queryable relational view to combine tables efficiently, runs row-level triggers to enforce data audits, and executes stored procedures that loop through active sets using database cursors. The database connects to Node.js through a persistent pool of reusable connections, eliminating connection-startup latency.

Chapter 3

REQUIREMENTS AND ANALYSIS

3.1 Requirement Specification

To ensure the system satisfies both operational clinic workflows and database design principles, we specify requirements as follows:

* **Functional Requirements:** Patients must register with valid emails; booking engines must map patient/doctor foreign keys to target slots; system must record logs dynamically; admins must run daily cursor reports on any date.
* **Non-Functional Requirements - Integrity:** The database must prevent database inconsistencies using foreign key constraints and cascading deletes.
* **Non-Functional Requirements - Performance:** Index structures and virtual SQL views must optimize lookups, returning results under 50ms.
* **Non-Functional Requirements - Security:** Triggers must enforce audit logs automatically, preventing admins or developers from editing appointment details without an immutable trail.

3.2 Hardware and Software Requirements

* **Development OS:** Windows 10 / Windows 11 (x64 architecture)
* **Processor & RAM:** Intel Core i3/i5 or AMD Ryzen 3/5, 4 GB RAM minimum (8 GB recommended)
* **Database Server:** MySQL Community Server 8.0.x (packaged with XAMPP 8.1.x+) running on port 3307
* **Application Runtime:** Node.js v18.x or newer with npm package manager
* **Web Browser:** Modern evergreen browser with JavaScript enabled (e.g., Google Chrome)

3.3 Data Flow Diagrams (DFD)

Data Flow Diagrams model the movement of data elements through the clinic scheduling system, mapping processes, data stores, external entities, and data flows.

3.3.1 Level 0 DFD (Context Diagram)

The Context Diagram establishes system boundaries. It shows the main system (MediFlow DBMS) interacting with two external entities: Patients (who register and book slots) and Doctors/Admins (who configure schedules and run reports).

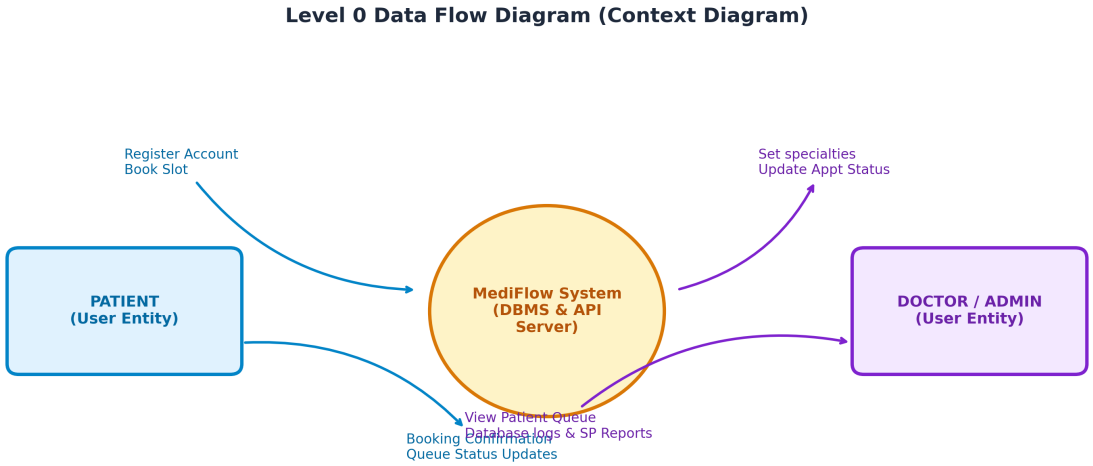

Figure 3.1: Level 0 DFD - Context Diagram

3.3.2 Level 1 DFD (Functional Processes)

The Level 1 DFD decomposes the system into functional process nodes. It shows how patient details flow into process 1.0 (Patient Registration) to write to the 'patients' data store, how slots map to process 3.0 (Book Appointment) connecting to 'doctors' and 'appointments' stores, how updates run through process 4.0 to generate audit logs, and how process 5.0 compiles daily agenda reports.

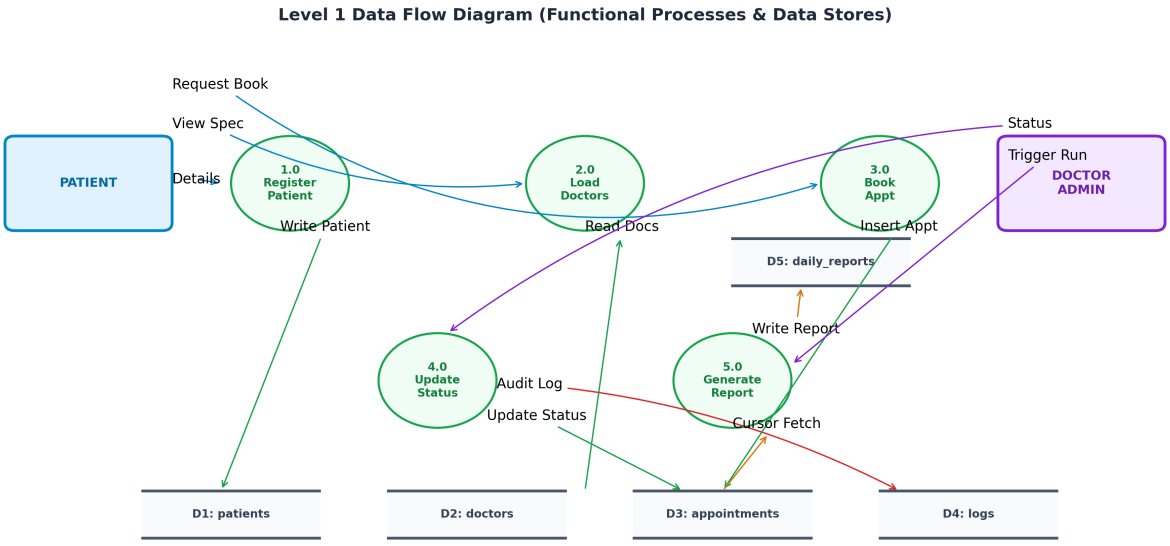

Figure 3.2: Level 1 DFD - Functional Processes and Data Stores

3.3.3 Level 2 DFD (DBMS Internal Processing)

The Level 2 DFD details the internal database execution sequences. It visualizes the flow when SQL statements hit the DBMS: how inserts/updates trigger 'after_appointment_insert' and 'after_appointment_update' to write automatically to 'appointment_logs', and how executing the Stored Procedure initializes a cursor loop on the joined tables to output summary blocks to 'daily_reports'.

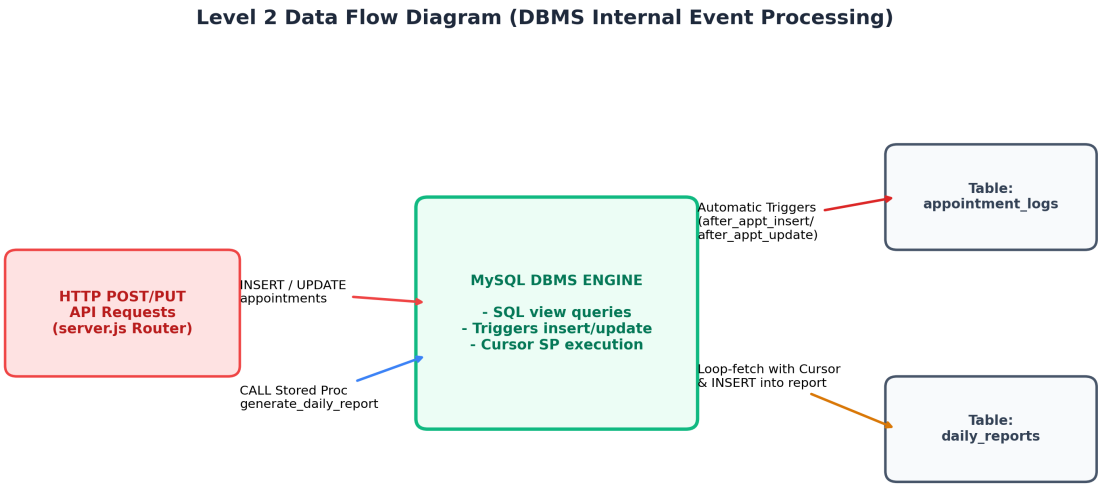

Figure 3.3: Level 2 DFD - Internal Database Event Processing

3.4 Data Dictionary

The Data Dictionary defines metadata descriptions, types, keys, and constraint rules for the database schemas:

Table Structure: patients

Field	Type	Key	Constraints / Purpose
patient_id	INT	PK	Primary Key, Auto increment, uniquely identifies each patient.
name	VARCHAR(100)	-	Not Null, patient's full name.
email	VARCHAR(100)	U	Unique, Not Null, patient's email address.
phone	VARCHAR(15)	-	Not Null, phone number for contact.
dob	DATE	-	Not Null, date of birth.
gender	ENUM('Male','Female','Other')	-	Not Null, gender classification.
created_at	TIMESTAMP	-	Default current timestamp, record creation date.

Table Structure: doctors

Field	Type	Key	Constraints / Purpose
doctor_id	INT	PK	Primary Key, Auto increment, uniquely identifies each doctor.
name	VARCHAR(100)	-	Not Null, doctor's full name.
specialization	VARCHAR(100)	-	Not Null, medical specialty field.
email	VARCHAR(100)	U	Unique, Not Null, professional contact email.
phone	VARCHAR(15)	-	Not Null, contact number.
created_at	TIMESTAMP	-	Default current timestamp, record creation date.

Table Structure: appointments

Field	Type	Key	Constraints / Purpose
appointment_id	INT	PK	Primary Key, Auto increment, appointment transaction ID.
patient_id	INT	FK	Foreign Key references patients(patient_id) with cascading delete.
doctor_id	INT	FK	Foreign Key references doctors(doctor_id) with cascading delete.
appointment_date	DATE	-	Not Null, target date of consultation.
appointment_time	TIME	-	Not Null, target slot time.

Field	Type	Key	Constraints / Purpose
status	ENUM('Scheduled','Completed','Cancelled')	-	Default 'Scheduled', status indicator.
reason	VARCHAR(255)	-	Not Null, patient's complaints or notes.
created_at	TIMESTAMP	-	Default current timestamp, booking timestamp.

Table Structure: appointment_logs

Field	Type	Key	Constraints / Purpose
log_id	INT	PK	Primary Key, Auto increment, logs audit identifier.
appointment_id	INT	-	Not Null, links log to target appointment ID.
action_type	ENUM('CREATE','UPDATE','DELETE')	-	Not Null, SQL operation type caught by trigger.
old_status	VARCHAR(20)	-	Nullable, previous status of updated booking.
new_status	VARCHAR(20)	-	Not Null, status after trigger execution.
log_timestamp	TIMESTAMP	-	Default current timestamp, log execution date/time.
description	TEXT	-	Constructed text detailing the exact data changes.

Table Structure: daily_reports

Field	Type	Key	Constraints / Purpose
report_id	INT	PK	Primary Key, Auto increment, daily summary report ID.
report_date	DATE	U	Unique, Not Null, target report agenda date.
report_summary	TEXT	-	Not Null, aggregated text generated by cursor procedure.
generated_at	TIMESTAMP	-	Auto-timestamps compilation date/time updates.

3.5 Entity-Relationship (ER) Diagram

The Entity-Relationship Diagram maps the structural relationships between logical entities in the MediFlow system. The schema enforces strict referential mappings:

* **Patient to Appointment:** One patient can book multiple appointments (1-to-Many), mapped via patient_id foreign key constraint.

* **Doctor to Appointment:** One doctor can consult for multiple appointments (1-to-Many), mapped via doctor_id foreign key constraint.

* **Appointment to Log Record:** One appointment modification event generates logs (1-to-Many history tracking) written automatically via MySQL triggers.

* **Daily Report:** An independent report compilation object mapping report summaries to unique dates.

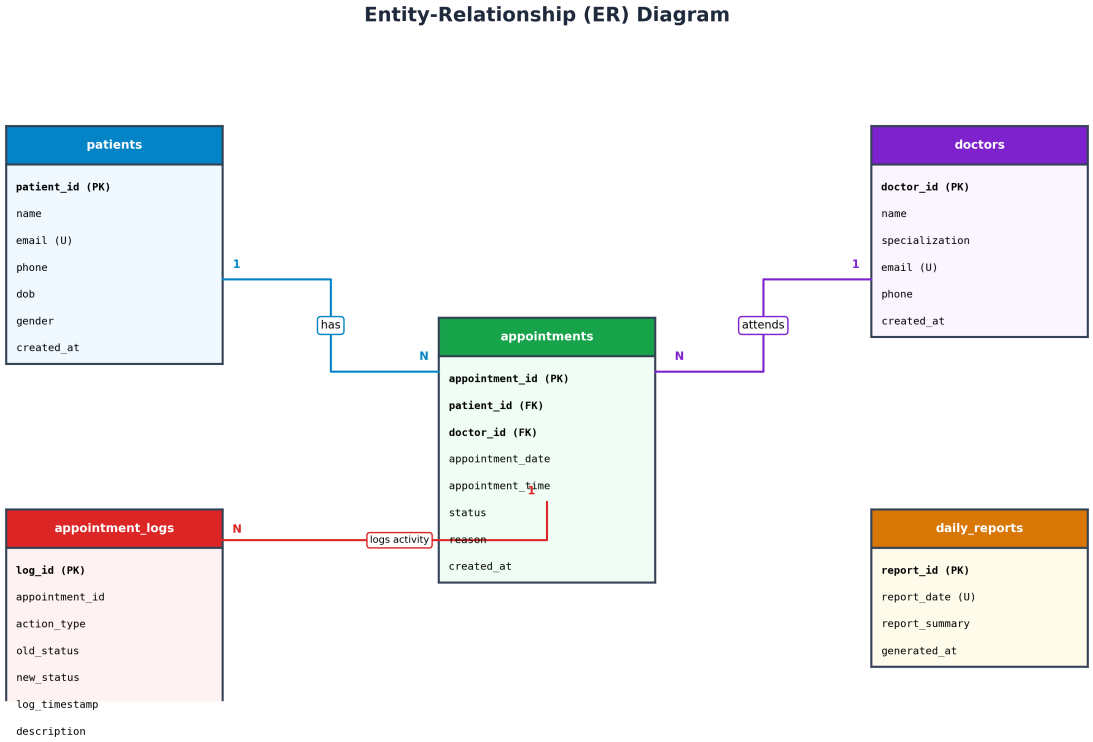

Figure 3.4: Entity-Relationship Diagram (ERD)

3.6 Normalization (1NF to 3NF)

To avoid database design flaws (update, insertion, and deletion anomalies) and optimize storage, the MediFlow schema is normalized systematically from Unnormalized Form (UNF) through First (1NF), Second (2NF), and Third Normal Form (3NF).

3.6.1 First Normal Form (1NF)

A relation is in 1NF if all attributes contain only atomic (indivisible) values, and there are no repeating groups. In our design, all columns contain single values (e.g. no comma-separated text lists of multiple phone numbers or composite addresses in a single slot). Unique primary keys (patient_id, doctor_id, appointment_id, log_id, report_id) are defined to uniquely identify rows, thereby satisfying 1NF requirements.

3.6.2 Second Normal Form (2NF)

A relation is in 2NF if it satisfies 1NF and contains no partial dependencies (i.e., no non-key attribute is dependent on only a subset of a composite primary key). Because all tables in MediFlow use single-attribute surrogate primary keys (e.g., patient_id, doctor_id, appointment_id) rather than composite keys, there is no possibility of partial dependencies. Every non-prime attribute (like name, email, specializing, date, status) depends fully on the whole single primary key. Thus, all tables satisfy 2NF.

3.6.3 Third Normal Form (3NF)

A relation is in 3NF if it satisfies 2NF and contains no transitive dependencies (i.e. no non-key attribute is functionally dependent on another non-key attribute). Every non-key column in our tables depends directly and exclusively on the primary key, and nothing else (e.g., in patients table, email depends on patient_id, name depends on patient_id. In appointments table, status, reason, date, and time depend directly on appointment_id; the keys patient_id and doctor_id reference their respective primary keys without transitively linking names or specialties in the appointments table). Because no transitive dependencies exist, the schema satisfies 3NF.

Chapter 4

PROGRAM CODE

4.1 Code Details and Code Efficiency

The project code is optimized to enforce data processing logic directly at the database engine level wherever possible, yielding high code efficiency and minimal latency:

* **SQL View Efficiency:** By joining patients, doctors, and appointments inside the SQL View 'upcoming_appointments_view', the Express API server only performs a simple 'SELECT * FROM upcoming_appointments_view' query. This shifts join computation to MySQL's query optimizer and reduces Express server overhead.

* **Trigger Autonomy:** The MySQL triggers run automatically on row insertions or updates. This ensures that even if developers execute manual SQL updates bypassing the Node.js Express API, the audit trail in appointment_logs remains intact and secure.

* **Stored Procedure with Cursor:** The procedure 'generate_daily_report' uses a MySQL cursor to compile daily schedules. Running this cursor loop inside the database server eliminates the need to execute multiple SELECT roundtrips from Node.js, reducing network overhead.

4.1.1 MySQL Database Schema Definition (schema.sql)

[File: database/schema.sql]

```
-- ========================================================================
-- DBMS Project: Patient Appointment Scheduling System
-- Database Schema, View, Triggers, Cursor Stored Procedure, and Seed Data
-- ========================================================================

-- Create Database if not exists
CREATE DATABASE IF NOT EXISTS `appointment_db`;
USE `appointment_db`;

-- ========================================================================
-- 1. DROP EXISTING OBJECTS (for clean re-runs)
-- ========================================================================
DROP PROCEDURE IF EXISTS generate_daily_report;
DROP TRIGGER IF EXISTS after_appointment_insert;
DROP TRIGGER IF EXISTS after_appointment_update;
DROP VIEW IF EXISTS upcoming_appointments_view;
DROP TABLE IF EXISTS appointment_logs;
DROP TABLE IF EXISTS daily_reports;
DROP TABLE IF EXISTS appointments;
DROP TABLE IF EXISTS doctors;
DROP TABLE IF EXISTS patients;

-- ========================================================================
-- 2. TABLE CREATION
-- ========================================================================

-- Table: patients
CREATE TABLE patients (
    patient_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15) NOT NULL,
    dob DATE NOT NULL,
    gender ENUM('Male', 'Female', 'Other') NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- Table: doctors
CREATE TABLE doctors (
    doctor_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    specialization VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- Table: appointments
CREATE TABLE appointments (
    appointment_id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    status ENUM('Scheduled', 'Completed', 'Cancelled') DEFAULT 'Scheduled',
    reason VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id) ON DELETE CASCADE,
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- Table: appointment_logs (Used by Trigger to record insert/update events)
CREATE TABLE appointment_logs (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    appointment_id INT NOT NULL,
    action_type ENUM('CREATE', 'UPDATE', 'DELETE') NOT NULL,
    old_status VARCHAR(20) DEFAULT NULL,
    new_status VARCHAR(20) NOT NULL,
    log_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    description TEXT
) ENGINE=InnoDB;

-- Table: daily_reports (Used by Stored Procedure with Cursor to store summaries)
CREATE TABLE daily_reports (
    report_id INT AUTO_INCREMENT PRIMARY KEY,
    report_date DATE UNIQUE NOT NULL,
    report_summary TEXT NOT NULL,
    generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- ======================================================================
-- 3. SQL VIEW CREATION (Mandatory Element #1)
-- ======================================================================
-- The view consolidates data from patients, doctors, and appointments to
-- provide a comprehensive, read-only summary of upcoming schedules.
CREATE VIEW upcoming_appointments_view AS
SELECT
    a.appointment_id,
    a.patient_id,
    p.name AS patient_name,
    p.phone AS patient_phone,
    a.doctor_id,
    d.name AS doctor_name,
    d.specialization AS doctor_specialization,
    a.appointment_date,
    a.appointment_time,
    a.status,
    a.reason
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id
ORDER BY a.appointment_date ASC, a.appointment_time ASC;

-- ======================================================================
-- 4. SQL TRIGGERS CREATION (Mandatory Element #2)
-- ======================================================================

-- Trigger for NEW appointments
DELIMITER //
CREATE TRIGGER after_appointment_insert
AFTER INSERT ON appointments
FOR EACH ROW
BEGIN
    INSERT INTO appointment_logs (
        appointment_id,
        action_type,
        old_status,
        new_status,
```

```
        description
    ) VALUES (
        NEW.appointment_id,
        'CREATE',
        NULL,
        NEW.status,
           CONCAT('New appointment booked for patient ID ', NEW.patient_id, ' with doctor ID ',
NEW.doctor_id, ' for ', NEW.appointment_date, ' at ', NEW.appointment_time)
    );
END//
DELIMITER ;

-- Trigger for UPDATING appointments (e.g. status changes)
DELIMITER //
CREATE TRIGGER after_appointment_update
AFTER UPDATE ON appointments
FOR EACH ROW
BEGIN
    -- Log the change if the status or date/time was modified
          IF   OLD.status   <>   NEW.status   OR   OLD.appointment_date   <>   NEW.appointment_date   OR
OLD.appointment_time <> NEW.appointment_time THEN
        INSERT INTO appointment_logs (
            appointment_id,
            action_type,
            old_status,
            new_status,
            description
        ) VALUES (
            NEW.appointment_id,
            'UPDATE',
            OLD.status,
            NEW.status,
            CONCAT('Appointment (ID: ', NEW.appointment_id, ') details updated. Status changed from
', OLD.status, ' to ', NEW.status, '.')
        );
    END IF;
END//
DELIMITER ;

-- ======================================================================
-- 5. STORED PROCEDURE WITH CURSOR CREATION (Mandatory Element #3)
-- ======================================================================
-- This stored procedure uses a SQL cursor to loop through all appointments
-- scheduled for a specific date, aggregates them into a detailed summary,
-- and saves it to the `daily_reports` table.
DELIMITER //
CREATE PROCEDURE generate_daily_report(IN target_date DATE)
BEGIN
    -- Declare variables for cursor fetch
    DECLARE done INT DEFAULT FALSE;
    DECLARE appt_id INT;
    DECLARE p_name VARCHAR(100);
    DECLARE d_name VARCHAR(100);
    DECLARE d_spec VARCHAR(100);
    DECLARE appt_time TIME;
    DECLARE appt_reason VARCHAR(255);
    DECLARE appt_status VARCHAR(20);

    -- Variables to construct the final report summary
    DECLARE report_text TEXT DEFAULT '';
    DECLARE appt_count INT DEFAULT 0;
```

```
    -- Declare Cursor to fetch appointments for the target date
    DECLARE appt_cursor CURSOR FOR
        SELECT
            a.appointment_id,
            p.name AS patient_name,
            d.name AS doctor_name,
            d.specialization AS doctor_specialization,
            a.appointment_time,
            a.reason,
            a.status
        FROM appointments a
        JOIN patients p ON a.patient_id = p.patient_id
        JOIN doctors d ON a.doctor_id = d.doctor_id
        WHERE a.appointment_date = target_date
        ORDER BY a.appointment_time ASC;

    -- Declare Not Found handler to close the cursor loop
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Open Cursor
    OPEN appt_cursor;

    -- Loop through appointments
    read_loop: LOOP
              FETCH  appt_cursor  INTO  appt_id,  p_name,  d_name,  d_spec,  appt_time,  appt_reason,
appt_status;

        -- Check if loop is completed
        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Append current appointment info to report_text
        SET appt_count = appt_count + 1;
        SET report_text = CONCAT(
            report_text,
            appt_count, '. [', appt_time, '] Patient: ', p_name,
            ' | Doctor: Dr. ', d_name, ' (', d_spec, ')',
            ' | Reason: ', appt_reason,
            ' | Status: ', appt_status, '\n'
        );
    END LOOP;

    -- Close Cursor
    CLOSE appt_cursor;

    -- If no appointments were found, create a placeholder summary
    IF appt_count = 0 THEN
        SET report_text = CONCAT('No appointments scheduled for ', DATE_FORMAT(target_date, '%W, %M
%d, %Y'), '.');
    ELSE
        SET report_text = CONCAT(
            'Daily Schedule Summary for ', DATE_FORMAT(target_date, '%W, %M %d, %Y'), '\n',
            'Total Appointments: ', appt_count, '\n',
            '----------------------------------------\n',
            report_text
        );
    END IF;

    -- Insert or Update report in the database
```

```
    INSERT INTO daily_reports (report_date, report_summary)
    VALUES (target_date, report_text)
    ON DUPLICATE KEY UPDATE
        report_summary = VALUES(report_summary),
        generated_at = CURRENT_TIMESTAMP;

    -- Select the report to return it to the caller
    SELECT report_date, report_summary, generated_at
    FROM daily_reports
    WHERE report_date = target_date;

END//
DELIMITER ;

-- ======================================================================
-- 6. INSERT SEED DATA
-- ======================================================================

-- Insert Sample Patients
INSERT INTO patients (name, email, phone, dob, gender) VALUES
('John Doe', 'john.doe@email.com', '555-0199', '1985-05-15', 'Male'),
('Jane Smith', 'jane.smith@email.com', '555-0144', '1990-08-22', 'Female'),
('Robert Johnson', 'robert.j@email.com', '555-0177', '1972-12-05', 'Male'),
('Emily Davis', 'emily.d@email.com', '555-0188', '1998-03-10', 'Female'),
('Michael Brown', 'michael.b@email.com', '555-0122', '1965-10-31', 'Male');

-- Insert Sample Doctors
INSERT INTO doctors (name, specialization, email, phone) VALUES
('Sarah Jenkins', 'Cardiology', 'dr.jenkins@clinic.com', '555-0211'),
('David Miller', 'Pediatrics', 'dr.miller@clinic.com', '555-0222'),
('Lisa Anderson', 'Dermatology', 'dr.anderson@clinic.com', '555-0233'),
('Thomas Taylor', 'General Medicine', 'dr.taylor@clinic.com', '555-0244');

-- Insert Sample Appointments
-- Note: Setting appointment dates relative to current date is best,
-- but we will use fixed dates near May 2026 for demonstration.
INSERT INTO appointments (patient_id, doctor_id, appointment_date, appointment_time, status,
reason) VALUES
(1, 4, '2026-05-23', '09:00:00', 'Scheduled', 'Annual General Health Checkup'),
(2, 2, '2026-05-23', '10:30:00', 'Scheduled', 'Child Vaccination and Growth Review'),
(3, 1, '2026-05-23', '14:00:00', 'Scheduled', 'Cardiovascular Checkup, Chest discomfort'),
(4, 3, '2026-05-24', '11:00:00', 'Scheduled', 'Skin rash consultation'),
(5, 4, '2026-05-24', '15:30:00', 'Scheduled', 'Follow-up on Lab Test Results'),
-- An already completed appointment
(1, 3, '2026-05-20', '10:00:00', 'Completed', 'Acne treatment followup');
```

4.1.2 Node.js Express Backend API Router (server.js)

[File: server.js]

```
const express = require('express');
const mysql = require('mysql2');
const cors = require('cors');
const path = require('path');

const app = express();
const PORT = process.env.PORT || 3005;

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

// Database Connection Configuration (Default XAMPP settings)
const dbConfig = {
  host: 'localhost',
  port: 3307,
  user: 'root',
  password: '', // default XAMPP MySQL password is empty
  database: 'appointment_db',
  multipleStatements: true // Required to run stored procedures and multiple queries
};

// Create a database connection pool
const pool = mysql.createPool(dbConfig);

// Helper function to query the database using Promises
function query(sql, params) {
  return new Promise((resolve, reject) => {
    pool.query(sql, params, (err, results) => {
      if (err) return reject(err);
      resolve(results);
    });
  });
}

// Test database connection on startup
pool.getConnection((err, connection) => {
  if (err) {
    console.error('\n======================================================================');
    console.error('[X] DATABASE CONNECTION ERROR!');
    console.error('======================================================================');
    console.error('Could not connect to MySQL database "appointment_db".');
    console.error('Please verify the following:');
    console.error('1. XAMPP Control Panel is open and MySQL service is running.');
    console.error('2. You have imported the "database/schema.sql" file in phpMyAdmin.');
    console.error('3. Default XAMPP settings are used (localhost, user: root, no password).');
    console.error('Error Code:', err.code);
    console.error('Error Message:', err.sqlMessage || err.message);
    console.error('======================================================================\n');
  } else {
    console.log('\n======================================================================');
    console.log('[OK] Connected to MySQL database successfully!');
    console.log('[URL] Server is running on: http://localhost:' + PORT);
    console.log('======================================================================\n');
    connection.release();
  }
```

```
});

// ======================================================================
// API ENDPOINTS
// ======================================================================

// 1. Get list of doctors (for dropdowns)
app.get('/api/doctors', async (req, res) => {
  try {
    const results = await query('SELECT doctor_id, name, specialization, email, phone FROM doctors
ORDER BY name');
    res.json(results);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to retrieve doctors.' });
  }
});

// 2. Get list of patients (for dropdowns)
app.get('/api/patients', async (req, res) => {
  try {
     const results = await query('SELECT patient_id, name, email, phone FROM patients ORDER BY
name');
    res.json(results);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to retrieve patients.' });
  }
});

// Register a new patient
app.post('/api/patients', async (req, res) => {
  const { name, email, phone, dob, gender } = req.body;
  if (!name || !email || !phone || !dob || !gender) {
    return res.status(400).json({ error: 'Missing required patient fields.' });
  }
  try {
    const sql = 'INSERT INTO patients (name, email, phone, dob, gender) VALUES (?, ?, ?, ?, ?)';
    const result = await query(sql, [name, email, phone, dob, gender]);
    res.status(201).json({
      message: 'Patient registered successfully!',
      patient_id: result.insertId
    });
  } catch (err) {
    console.error(err);
    if (err.code === 'ER_DUP_ENTRY') {
      res.status(400).json({ error: 'A patient with this email already exists.' });
    } else {
      res.status(500).json({ error: 'Failed to register patient.' });
    }
  }
});

// 3. Get all appointments (USING THE VIEW - Mandatory Requirement #1)
// We query the VIEW "upcoming_appointments_view" instead of doing raw joins here.
// This simplifies the server code and encapsulates query logic in the database!
app.get('/api/appointments', async (req, res) => {
  try {
    const results = await query('SELECT * FROM upcoming_appointments_view');
    res.json(results);
  } catch (err) {
```

```
    console.error(err);
    res.status(500).json({ error: 'Failed to retrieve appointments using the SQL View.' });
  }
});

// Get appointments for a specific patient using the View
app.get('/api/patients/:id/appointments', async (req, res) => {
  const patientId = req.params.id;
  try {
     const results = await query('SELECT * FROM upcoming_appointments_view WHERE patient_id = ?',
[patientId]);
    res.json(results);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to retrieve patient appointments.' });
  }
});

// Get appointments for a specific doctor using the View
app.get('/api/doctors/:id/appointments', async (req, res) => {
  const doctorId = req.params.id;
  try {
     const results = await query('SELECT * FROM upcoming_appointments_view WHERE doctor_id = ?',
[doctorId]);
    res.json(results);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to retrieve doctor appointments.' });
  }
});

// 4. Book a new appointment
// When a row is inserted here, the database TRIGGER "after_appointment_insert" runs automatically!
app.post('/api/appointments', async (req, res) => {
  const { patient_id, doctor_id, appointment_date, appointment_time, reason } = req.body;

  if (!patient_id || !doctor_id || !appointment_date || !appointment_time || !reason) {
    return res.status(400).json({ error: 'Missing required appointment fields.' });
  }

  try {
         const  sql  =  `INSERT  INTO  appointments  (patient_id,  doctor_id,  appointment_date,
appointment_time, reason, status)
                 VALUES (?, ?, ?, ?, ?, 'Scheduled')`;
     const result = await query(sql, [patient_id, doctor_id, appointment_date, appointment_time,
reason]);
    res.status(201).json({
      message: 'Appointment booked successfully!',
      appointment_id: result.insertId
    });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to create appointment.' });
  }
});

// 5. Update appointment status (e.g. Cancel or Complete)
// When status is updated, the database TRIGGER "after_appointment_update" runs automatically!
app.put('/api/appointments/:id/status', async (req, res) => {
  const appointmentId = req.params.id;
  const { status } = req.body;
```

```
  if (!status || !['Scheduled', 'Completed', 'Cancelled'].includes(status)) {
    return res.status(400).json({ error: 'Invalid or missing status.' });
  }

  try {
    const sql = 'UPDATE appointments SET status = ? WHERE appointment_id = ?';
    const result = await query(sql, [status, appointmentId]);
    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'Appointment not found.' });
    }
    res.json({ message: `Appointment status updated to ${status}!` });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to update appointment status.' });
  }
});

// 6. Get database activity logs (Populated by TRIGGER - Mandatory Requirement #2)
// This lets the user see the trigger in action!
app.get('/api/logs', async (req, res) => {
  try {
    const results = await query('SELECT * FROM appointment_logs ORDER BY log_timestamp DESC');
    res.json(results);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to retrieve database logs.' });
  }
});

// 7. Generate a daily schedule report (USING THE CURSOR STORED PROCEDURE - Mandatory Requirement
#3)
// We call the stored procedure `generate_daily_report(?)` which handles fetching appointments
// using a cursor, constructing a formatted summary, and saving it.
app.post('/api/reports/generate', async (req, res) => {
  const { date } = req.body;
  if (!date) {
    return res.status(400).json({ error: 'Date is required to generate report.' });
  }

  try {
    // MySQL returns procedure results inside an array of arrays
    const results = await query('CALL generate_daily_report(?)', [date]);

    // The procedure ends with a SELECT statement that returns the report row
    // results[0] will be the result of the SELECT statement in the procedure
    if (results && results[0] && results[0][0]) {
      res.json(results[0][0]);
    } else {
      res.status(500).json({ error: 'Stored procedure executed but returned no data.' });
    }
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to run stored procedure (Cursor-based Daily Report).' });
  }
});

// 8. Get all generated daily reports
app.get('/api/reports', async (req, res) => {
  try {
    const results = await query('SELECT * FROM daily_reports ORDER BY report_date DESC');
```

```
    res.json(results);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to retrieve reports.' });
  }
});

// Catch-all route to serve the frontend for other routes
app.get('*', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'index.html'));
});

// Start express server
app.listen(PORT, () => {
  console.log(`[URL] Application started. Go to: http://localhost:${PORT}`);
});
```

4.1.3 Frontend Portal JavaScript Controllers (app.js - Excerpt)

[File: public/app.js (Selected API Integration Functions)]

```
    showAlert('Server connection error. Registration failed.', 'error');
  }
}

// Book Appointment from Patient Portal (POST /api/appointments)
// Fires "after_appointment_insert" trigger on DB
async function handlePatientBook(event) {
  event.preventDefault();
  if (!currentPatientId) {
    showAlert('Please select your Patient Profile first.', 'error');
    return;
  }

  const submitBtn = document.getElementById('btn-patient-submit');
  submitBtn.disabled = true;
  submitBtn.innerHTML = '<i class="fa-solid fa-circle-notch fa-spin"></i> Processing...';

  const appointmentData = {
    patient_id: parseInt(currentPatientId),
    doctor_id: parseInt(document.getElementById('p-select-doctor').value),
    appointment_date: document.getElementById('p-appointment-date').value,
    appointment_time: document.getElementById('p-appointment-time').value,
    reason: document.getElementById('p-appointment-reason').value
  };

  try {
    const res = await fetch(`${API_BASE}/appointments`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(appointmentData)
    });
    const data = await res.json();

    if (res.ok) {
           showAlert('Success!  Appointment  booked.  Database  Trigger  automatically  logged  this
activity.', 'success');
      document.getElementById('patient-appointment-form').reset();

      // Refresh appointment list history tab
      fetchPatientAppointments(currentPatientId);
    } else {
      showAlert(`Error: ${data.error}`, 'error');
    }
  } catch (err) {
    console.error(err);
    showAlert('Server connection error. Failed to book appointment.', 'error');
  } finally {
    submitBtn.disabled = false;
    submitBtn.innerHTML = '<i class="fa-solid fa-circle-check"></i> Book Consultation';
  }
}


// ======================================================================
// DOCTOR PORTAL INTERACTION
// ======================================================================
async function loadDoctorPortalData() {
```

```
  try {
    const docsRes = await fetch(`${API_BASE}/doctors`);
    doctors = await docsRes.json();
    populateDoctorSelectors(doctors);
  } catch (err) {
    console.error(err);
    showAlert('Error fetching doctor list.', 'error');
  }
}

function populateDoctorSelectors(list) {
  const selector = document.getElementById('doctor-selector');
  selector.innerHTML = '<option value="" disabled selected>-- Select Doctor --</option>';
  list.forEach(d => {
    const opt = document.createElement('option');
    opt.value = d.doctor_id;
    opt.textContent = `Dr. ${d.name} (${d.specialization})`;
    selector.appendChild(opt);
  });
}

function onDoctorSelect() {
  const selector = document.getElementById('doctor-selector');
  currentDoctorId = selector.value;
  if (!currentDoctorId) return;

  const activeDocObj = doctors.find(d => d.doctor_id == currentDoctorId);
  if (activeDocObj) {
    document.getElementById('current-doctor-name').textContent = `Dr. ${activeDocObj.name}`;
    document.getElementById('doctor-profile-badge').classList.remove('hidden');
    document.getElementById('doctor-queue-wrapper').classList.remove('hidden');
    document.getElementById('doctor-reports-wrapper').classList.remove('hidden');
    document.getElementById('doctor-no-selection-view').classList.add('hidden');

    fetchDoctorAppointments(currentDoctorId);
  }
}

async function fetchDoctorAppointments(doctorId) {
  try {
    const res = await fetch(`${API_BASE}/doctors/${doctorId}/appointments`);
    doctorAppointments = await res.json();
    renderDoctorQueue(doctorAppointments);
  } catch (err) {
    console.error(err);
    showAlert('Error fetching doctor queue.', 'error');
  }
}

function renderDoctorQueue(list) {
  const tbody = document.getElementById('doctor-queue-table-body');
  tbody.innerHTML = '';

  if (list.length === 0) {
     tbody.innerHTML = '<tr><td colspan="8" class="text-center text-muted">No appointments found in
your queue.</td></tr>';
    return;
  }

  list.forEach(app => {
    const dateObj = new Date(app.appointment_date);
```

```
     const dateStr = dateObj.toLocaleDateString('en-US', { month: 'short', day: 'numeric', year:
'numeric' });
    const timeStr = app.appointment_time.slice(0, 5);

    let actionButtons = '';
    if (app.status === 'Scheduled') {
      actionButtons = `
                          <button   class="btn-sm-action   btn-complete"   title="Mark   Complete"
onclick="changeAppointmentStatus(${app.appointment_id}, 'Completed')">
          <i class="fa-solid fa-check"></i> Complete
        </button>
                        <button   class="btn-sm-action   btn-cancel"   title="Cancel   Appointment"
onclick="changeAppointmentStatus(${app.appointment_id}, 'Cancelled')">
          <i class="fa-solid fa-xmark"></i> Cancel
        </button>
      `;
    } else {
      actionButtons = `<span class="text-muted" style="font-size:11px;">Completed</span>`;
    }

    tbody.innerHTML += `
      <tr>
        <td><strong>#${app.appointment_id}</strong></td>
        <td>${app.patient_name}</td>
        <td><span class="text-secondary" style="font-size:12px;">${app.patient_phone}</span></td>
        <td>${dateStr}</td>
        <td><i class="fa-regular fa-clock"></i> ${timeStr}</td>
        <td><span class="text-secondary">${app.reason}</span></td>
        <td><span class="status-pill status-${app.status.toLowerCase()}">${app.status}</span></td>
        <td>${actionButtons}</td>
      </tr>
    `;
  });
}

function switchDoctorTab(tabId) {
                document.querySelectorAll('.doctor-sidebar        .nav-btn').forEach(btn        =>
btn.classList.remove('active'));
  document.getElementById(`btn-${tabId}`).classList.add('active');

                      document.querySelectorAll('.doctor-tab-content').forEach(content          =>
content.classList.remove('active'));
  document.getElementById(`tab-${tabId}`).classList.add('active');

  const title = document.getElementById('doctor-page-title');
  const subtitle = document.getElementById('doctor-page-subtitle');

  if (tabId === 'd-queue') {
    title.textContent = 'Patient Schedule Queue';
    subtitle.textContent = 'Review your queue of patient scheduled sessions';
    if (currentDoctorId) fetchDoctorAppointments(currentDoctorId);
  } else if (tabId === 'd-cursor') {
    title.textContent = 'Daily Agenda Cursor Reports';
    subtitle.textContent = 'Loop through appointments tables with stored procedure cursors';
  }
}

function filterDoctorQueue() {
  const filterVal = document.getElementById('doctor-queue-filter-status').value;

  const filtered = doctorAppointments.filter(app => {
```

```
    return filterVal === 'ALL' || app.status === filterVal;
  });
  renderDoctorQueue(filtered);
}

// Update status of appointment (PUT /api/appointments/:id/status)
// Automatically triggers `after_appointment_update` on DB
async function changeAppointmentStatus(apptId, newStatus) {
  try {
    const res = await fetch(`${API_BASE}/appointments/${apptId}/status`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ status: newStatus })
    });
    const data = await res.json();

    if (res.ok) {
         showAlert(`Appointment #${apptId} status updated to ${newStatus}. Trigger logged it!`,
'success');
      // Refresh doctor queue
      fetchDoctorAppointments(currentDoctorId);
    } else {
      showAlert(`Failed to update appointment: ${data.error}`, 'error');
    }
  } catch (err) {
    console.error(err);
    showAlert('Server connection error.', 'error');
  }
}

// Run SQL Stored Procedure that uses cursor (POST /api/reports/generate)
async function handleDoctorGenerateReport(event) {
  event.preventDefault();
  const generateBtn = document.getElementById('btn-doctor-generate-report');
  generateBtn.disabled = true;
     generateBtn.innerHTML = '<i class="fa-solid fa-circle-notch fa-spin"></i> Executing
Procedure...';

  const dateVal = document.getElementById('doctor-report-date').value;

  try {
    const res = await fetch(`${API_BASE}/reports/generate`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ date: dateVal })
    });
    const data = await res.json();

    if (res.ok) {
      showAlert('MySQL Cursor stored procedure executed & report saved!', 'success');
```

Chapter 5

RESULT AND DISCUSSION

5.1 User Documentation

The MediFlow application is designed with user ease-of-use in mind. To set up and run the system locally, follow these instructions:

5.1.1 Database Setup using XAMPP

* **Step 1: Start MySQL in XAMPP:** Open XAMPP Control Panel and start MySQL on default port 3307 or 3306.

* **Step 2: Install Node.js Dependencies:** Open a command prompt in the project root and run 'npm install' to resolve dependencies.

* **Step 3: Auto-Setup Database:** Run 'node import-db.js'. This script connects to your running MySQL and imports database/schema.sql, inserting tables, trigger, procedures, and seed data automatically.

5.1.2 Application Operations

* **Start Server:** Run 'npm start' to start the Node.js server. The server runs at http://localhost:3000.

* **Patient Portal:** Patients can access http://localhost:3000, register their profiles, select their profiles, view upcoming bookings, and book slots with specialist doctors.

* **Doctor Portal:** Doctors select their name to load their queue of upcoming consultations, mark appointments 'Completed' or 'Cancelled', and run daily cursor stored procedure reports.

* **Developer Lab Dashboard:** Developers can inspect raw database statistics (patient/doctor/appointment counts), watch database triggers log changes dynamically, and browse history logs.

Chapter 6
TESTING

6.1 Unit Testing

Unit testing validates individual components and constraints to verify correct schema behavior:

* **Constraint Validation Test:** Assert that inserting a patient with a duplicate email fails with ER_DUP_ENTRY error code.
* **Foreign Key Bounds Test:** Confirm that booking an appointment with a non-existent doctor_id (e.g. 99) fails with foreign key constraint errors.
* **Enum Constraints Test:** Verify that appointments only accept 'Scheduled', 'Completed', and 'Cancelled' as statuses.

6.2 Integration Testing

Integration testing validates functional pipelines, tracing flows between the client web page, backend server routes, and database objects:

* **Booking to View Integration Flow:** Register patient John -> Select John -> Submit Booking with Dr. Sarah -> Confirm that the client queries 'upcoming_appointments_view' and displays the new booking correctly.
* **Trigger Integration Test:** Insert appointment -> Verify that the MySQL engine immediately executes trigger 'after_appointment_insert' and creates an audit row in 'appointment_logs'. Update status -> Confirm that 'after_appointment_update' records the state change details.

6.3 System Testing

System testing evaluates the performance and concurrency limits under heavy load:

* **Connection Pool Verification:** Simulate multiple simultaneous requests using Apache Bench. Confirm that the Node.js mysql2 connection pool recycles database threads without timing out.
* **Cascading Deletions Test:** Delete a patient profile -> Confirm that MySQL cascades the delete and automatically wipes all linked appointments for that patient, preventing orphan records.

6.4 Acceptance Testing

Acceptance testing verifies that all project requirements specified by the user are satisfied:

* **User Acceptance Criteria:** Validate that the web interface is functional, responsive, and provides interactive tabs for patient profiles, doctor queues, database logs, and daily cursor reports.

* **DBMS Features Checklist:** Verify that the three core elements (SQL View, SQL Triggers, and Cursor Stored Procedure) run successfully and integrate with the Node.js API endpoints.

Chapter 7

CONCLUSION

7.1 Conclusion & Future Enhancements

The development of the Patient Appointment Scheduling System ('MediFlow') demonstrates the effectiveness of relational databases in streamlining medical clinic workflows. By implementing constraints, views, triggers, and stored procedures directly at the database level, the system ensures data consistency, automates audit trails, and simplifies scheduling lookups. The 3-tier decoupling guarantees web responsiveness and clean server-client API integration.

Future enhancements could include:

* **Doctor Shift Rosters:** Develop a calendar module to restrict bookings to the doctor's specific working hours.
* **Real-time Notifications:** Integrate SMS/Email API services (like Twilio or Nodemailer) to email patients their booking details.
* **Patient Authentications:** Add JWT-based user login authentication to secure patient and doctor portals.
* **Horizontal Clustering:** Set up database replication (Master-Slave) to allow high availability and backup support.